

Workshop Implementation of Software Engineering Design

WISED – Prof. Dr. Azhari, MT

April 9th, 2026

Source Code [Link](#)

1. INTERNAL MANAGEMENT — BOARD PERIOD MANAGEMENT

KELOMPOK PANITIA & KOMITE (COMMITTEE/BOARD)

Fitur: Board Period Management

Spesifikasi

- (a) **Story Backlog:** "Sebagai Komite, saya ingin sistem secara otomatis menonaktifkan status pengurus yang masa jabatannya sudah berakhir."
- (b) **Deskripsi Teknis:** Melakukan perbandingan logika antara kolom end_year pada tabel alumni_boards dengan tahun sistem saat ini.
- (c) **Output UI/UX:** Label status visual (Hijau untuk "Aktif" dan Abu-abu untuk "Demisioner") pada tabel daftar pengurus.

Soal Ujian (Menu & Logic)

Tambahkan item menu baru pada sidebar kiri: **"Manajemen Internal"**. Buatlah sebuah tombol fungsi di dalam halaman tersebut yang jika diklik akan mengeksekusi perintah UPDATE massal untuk mengubah is_active = 0 pada semua record yang memiliki nilai end_year kurang dari 2026, dan menampilkan daftar seluruh pengurus baru dan lama.

SCREENSHOTS FROM STREAMLIT

Menu

- Pilih fitur
- Manajemen Internal
- Audit Transaksi
- Tiket & Order

Database: alumnievents.db
SQL source: uts_dump.sql
Auto init aktif database akan dibuat otomatis jika belum ada.

Alumni Events Dashboard

Manajemen Internal

Board Period Management — sinkronisasi status pengurus lama dan tampilan daftar pengurus dengan label visual.

Sinkronkan status demisioner (< 2026) Logika ujian dijalankan sesuai instruksi: semua pengurus dengan end_year < 2026 akan di-set is_active = 0.

Filter status: Semua (50 rows per page)

Filter posisi: Choose options

Total Pengurus: 15
Aktif: 5
Demisioner: 10

Halaman: 1

Menampilkan baris 1-15 dari 15

Board ID	Nama	Jabatan	Mulai	Selesai	Status	Email
15	Adhitya Santoso Yulianto	Head of E-Commerce	2024	2027	Aktif	adhitya.santoso33@alumni.ugm.ac.id
14	Bunga Setiawan	Head of Events	2024	2027	Aktif	bunga.setiawan17@alumni.ugm.ac.id
11	Ahmad Sanjaya Sasmita	President	2024	2027	Aktif	ahmad.sanjaya36@alumni.ugm.ac.id
12	Ahmad Ramadhan Nasution	Secretary	2024	2027	Aktif	ahmad.ramadhan38@alumni.ugm.ac.id
13	Bunga Lestari	Treasurer	2024	2027	Aktif	bunga.lestari44@alumni.ugm.ac.id
10	Bunga Mulyani Hermawan	Head of E-Commerce	2021	2024	Demisioner	bunga.mulyani48@alumni.ugm.ac.id
9	Adhitya Baskoro	Head of Events	2021	2024	Demisioner	adhitya.baskoro42@alumni.ugm.ac.id
6	Adhitya Purnama	President	2021	2024	Demisioner	adhitya.purnama56@alumni.ugm.ac.id
7	Ahmad Wibowo	Secretary	2021	2024	Demisioner	ahmad.wibowo60@alumni.ugm.ac.id
8	Ahmad Hartono	Treasurer	2021	2024	Demisioner	ahmad.hartono20@alumni.ugm.ac.id

DESCRIPTION

This feature was developed to automate the management of committee and board member status based on their term period. The main goal is to ensure that members whose term has already ended are no longer marked as active in the system.

From a business perspective, this solves the problem of outdated organizational data. Instead of manually checking one member at a time, the system can perform a mass update based on the `end_year` field stored in the `alumni_boards` table. If a board member's `end_year` is earlier than 2026, the system changes the `is_active` field to 0, which means the member is no longer active.

In the code, this feature is implemented inside the "Internal Management" menu in Streamlit. When the user opens the page, the application first connects to the local database and retrieves board data by joining `alumni_boards` with `alumni_users`, so that the table can display not only structural data such as position and year period, but also the related member name and email. The displayed table includes both current and previous board members.

```
with sqlite3.connect(DB_PATH) as conn:
    print("== Preview 1: Board Management ==")
    display(pd.read_sql_query(
        """
        SELECT
            ab.board_id,
            au.full_name,
            ab.position,
            ab.start_year,
            ab.end_year,
            CASE WHEN ab.is_active = 1 THEN 'Aktif' ELSE 'Demisioner' END AS status
        FROM alumni_boards ab
        JOIN alumni_users au ON au.user_id = ab.user_id
        ORDER BY ab.end_year DESC, ab.position ASC
        """,
        conn
    ))
```

A button is provided to execute the mass update process. When the button is clicked, the code runs an SQL UPDATE statement that sets `is_active = 0` for all rows where `end_year < 2026`. After the update is committed, the application immediately reloads the board table so the user can see the latest status without restarting the app.

Manajemen Internal

Board Period Management — sinkronisasi status pengurus lama dan tampilkan daftar pengurus dengan label visual.

Sinkronkan status demisioner (<2026)

Logika ujian dijalankan sesuai instruksi: semua pengurus dengan `end_year < 2026` akan di-set `is_active = 0`.

Filter status

Semua

Filter posisi

Choose options

Rows per page

50

Total Pengurus

15

Aktif

5

Demisioner

10

Sinkronkan status demisioner (< 2026)

To improve readability, the table also includes a visual status label. Members who are still active are shown with a green label, while members whose term has ended are shown with a gray label marked as “Demissioner.” This visual distinction makes it easier for users to interpret the result of the update.

Board ID	Nama	Jabatan	Mulai	Selesai	Status	Email
15	Adhitya Santoso Yulianto	Head of E-Commerce	2024	2027	Aktif	adhitya.santoso33@alumni.ugm.ac.id
14	Bunga Setiawan	Head of Events	2024	2027	Aktif	bunga.setiawan17@alumni.ugm.ac.id
11	Ahmad Sanjaya Sasmita	President	2024	2027	Aktif	ahmad.sanjaya36@alumni.ugm.ac.id
12	Ahmad Ramadhan Nasution	Secretary	2024	2027	Aktif	ahmad.ramadhan38@alumni.ugm.ac.id
13	Bunga Lestari	Treasurer	2024	2027	Aktif	bunga.lestari46@alumni.ugm.ac.id
10	Bunga Mulyani Hermawan	Head of E-Commerce	2021	2024	Demissioner	bunga.mulyani68@alumni.ugm.ac.id
9	Adhitya Baskoro	Head of Events	2021	2024	Demissioner	adhitya.baskoro42@alumni.ugm.ac.id
6	Adhitya Purnama	President	2021	2024	Demissioner	adhitya.purnama56@alumni.ugm.ac.id
7	Ahmad Wibowo	Secretary	2021	2024	Demissioner	ahmad.wibowo60@alumni.ugm.ac.id
8	Ahmad Hartono	Treasurer	2021	2024	Demissioner	ahmad.hartono20@alumni.ugm.ac.id

Overall, this feature combines simple business logic, direct database manipulation, and clear UI feedback. It demonstrates how Streamlit can be used not only to display data, but also to manage and update operational records interactively.

2. TRANSACTION AUDIT — ORDER TRACEABILITY AUDIT

KELOMPOK ADMINISTRATOR (SYSTEM ADMIN)

Fitur: Order Traceability Audit

Spesifikasi

- (a) **Story Backlog:** "Sebagai Admin, saya ingin melihat sejarah lengkap status pesanan jika terdapat komplain terkait kegagalan mesin vending", menurut setiap event acara alumni.
- (b) **Deskripsi Teknis:** Menggunakan JOIN antara tabel merch_orders dan merch_order_status_history berdasarkan order_id, dan tabel event.
- (c) **Output UI/UX:** Tampilan *Timeline View* yang merinci setiap perubahan status beserta stempel waktu (timestamp) kejadiannya.

Soal Ujian (Menu & Logic)

Tambahkan item menu baru pada sidebar kiri: "**Audit Transaksi**". Buatlah sebuah form input untuk order_id. Jika disubmit, program harus memanggil seluruh histori status dari tabel merch_order_status_history dan menampilkannya secara kronologis.

SCREENSHOTS FROM STREAMLIT

The screenshot displays the 'Alumni Events Dashboard' with a sidebar on the left containing navigation links like 'Home', 'Events', 'Orders', and 'Audit Transaksi'. The main content area is titled 'Audit Transaksi' and includes a search bar for 'Order ID'. Below the search bar, there is a table of order status history. The table has columns for Order ID, Order Date, Pickup Time, Total Price, Status, VM, Nama, Event, and Voucher Proof. The table shows 10 rows of data, with the 10th row (Order ID 11942) highlighted in red. Below the table, there is a section for 'Pilih order untuk audit' with a dropdown menu showing '11942'. The selected order details are shown below, including the order status 'Completed', total price 'Rp 3,049,869', and VM 'VM-03'. The 'Timeline history status' section shows a single status change from 'Pending' to 'Completed' at 2027-11-19 11:00:00. The 'Bukti vouching' section shows a photo of a person.

The screenshot displays the 'Audit Transaksi' section of the dashboard. It includes a search bar for 'Order ID' and a dropdown menu for 'Filter status order' set to 'Semua'. Below the search bar, there is a table of order status history. The table has columns for Order ID, Order Date, Pickup Time, Total Price, Status, VM, Nama, Event, and Voucher Proof. The table shows 10 rows of data, with the 10th row (Order ID 11942) highlighted in red. Below the table, there is a section for 'Pilih order untuk audit' with a dropdown menu showing '11942'. The selected order details are shown below, including the order status 'Completed', total price 'Rp 3,049,869', and VM 'VM-03'. The 'Timeline history status' section shows a single status change from 'Pending' to 'Completed' at 2027-11-19 11:00:00. The 'Bukti vouching' section shows a photo of a person.

DESCRIPTION

This feature was created to help administrators investigate the full history of an order, especially when there is a complaint related to the vending machine, merchandise pickup, or transaction flow during an alumni event. The purpose is to provide traceability, meaning that every important status change of an order can be tracked chronologically.

The data for this feature comes from several related tables, mainly `merch_orders`, `merch_order_status_history`, and `events`. The `merch_orders` table stores the main order information, the `merch_order_status_history` table stores every status update associated with an order, and the `events` table provides the event context so the admin can identify which event the order belongs to.

In the code, this feature is implemented in the “Transaction Audit” menu. The app provides two ways to access data: the user can either manually input an `order_id`, or choose from a selectable list of recent orders shown in the interface. This avoids the inconvenience of typing random IDs and makes the feature easier to use, especially during demonstrations.



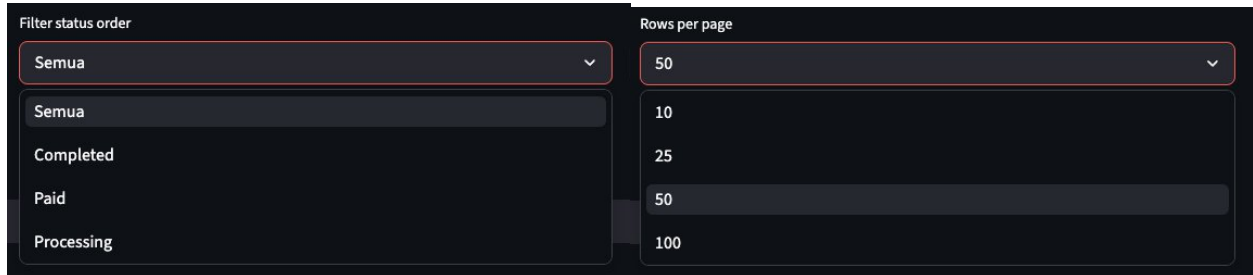
Once an order is selected and submitted, the application runs a query that joins the order table with its status history and related event information. The results are then sorted by timestamp in ascending order so the status flow appears in the correct chronological sequence. This is important because the main purpose of the feature is not only to show the current state of the order, but to explain how the order reached that state over time.

On the user interface, the result is presented as a timeline-style display. Each row or section shows the status name and the exact timestamp of the change. Additional details such as event name and order metadata can also be shown to give better context. This allows an administrator to see whether an order was created, processed, paid, packaged, completed, or failed, and in what order those events occurred.

Order ID	Status Akhir	Total Harga	Vending Machine
11942	Completed	Rp 3.049.869	VM-03
<pre>{ "Nama": "Hasan Puspita Widjojo" "Email": "hasan.puspita47@alumni.uign.ac.id" "Event": "Grand Alumni Homecoming 2027" "Tanggal Event": "2027-11-20 18:00:00" "Order Date": "2027-11-19 11:00:00" "Pickup Time": "2027-11-20 23:00:00" }</pre>			
Timeline histori status <>			
1. New Order → Paid			
2027-11-19 11:00:00			
Sistem: Pembayaran berhasil dikonfirmasi			
2. Paid → Completed			
2027-11-20 16:00:00			
Sistem: Barang berhasil diambil di VM-03			

Daffa Maulana Siddiq
24/533358/PA/22569

In terms of implementation, the code separates the responsibilities clearly: one part handles database access, one part handles user input from Streamlit widgets, and one part formats the returned data for presentation. This makes the feature easier to maintain and extend later. For example, filters, pagination, or downloadable audit logs could be added with minimal changes.



The image shows a dark-themed user interface with two side-by-side dropdown menus. The left menu is titled 'Filter status order' and has a dropdown arrow. Below the arrow, a list of options is visible: 'Semua', 'Completed', 'Paid', and 'Processing'. The right menu is titled 'Rows per page' and also has a dropdown arrow. Below the arrow, a list of options is visible: '10', '25', '50', and '100'. The '50' option is currently selected and highlighted with a dark background.

This feature is useful because it turns raw historical records into a readable operational audit trail. Instead of forcing the admin to inspect multiple tables manually, the dashboard presents the complete order journey in a single view.

3. TICKETS & ORDERS — SELF-SERVICE QR TICKET AND PICKUP

KELOMPOK PENJUAL & PESERTA (SELLERS & PARTICIPANTS)

Fitur: Self-Service QR Ticket & Pickup

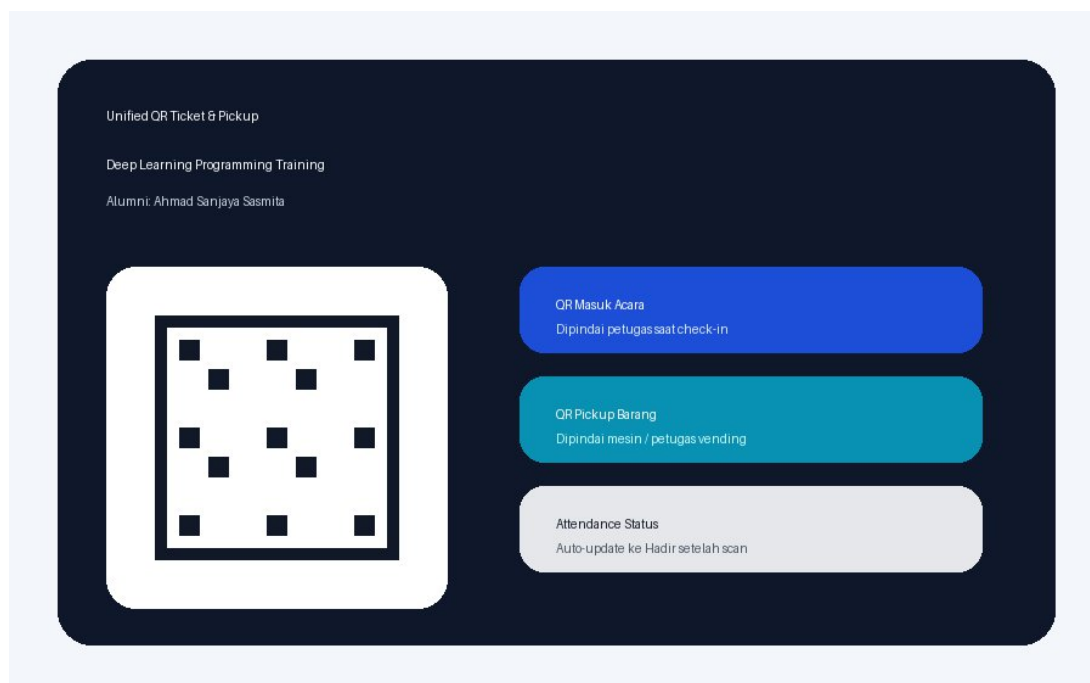
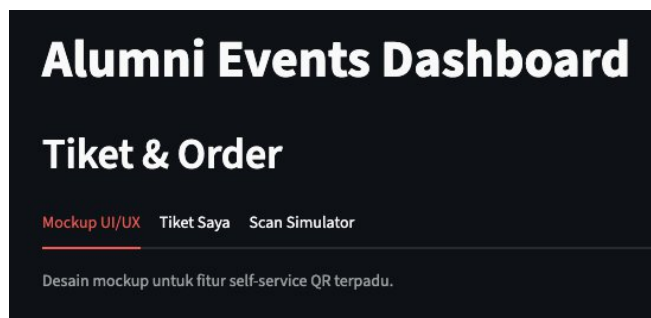
Spesifikasi

- (a) **Story Backlog:** "Sebagai Alumni, saya ingin memiliki satu kode QR terpadu yang berfungsi sebagai tiket masuk acara sekaligus kode pengambilan barang."
- (b) **Deskripsi Teknis:** Query data pada kolom qr_code di tabel event_participants dan order_id pada tabel merch_orders, dan tabel event,
- (c) **Output UI/UX:** Tab menu "Tiket Saya" yang menampilkan kode QR dinamis untuk dipindai oleh petugas atau mesin, menurut masing-masing event kegiatan.

Soal Ujian (Menu & Logic)

Buatlah sebuah desain gambar mockup UI/UX berdasarkan spesifikasi fitur diatas. Kemudian implementasikan ke dalam kode program disertai tambahan item menu baru pada sidebar kiri: **"Tiket & Order"**. sebuah fungsi yang mengambil qr_code unik milik user. Tambahkan logika simulasi: jika kode tersebut di-input ke sistem, program harus memperbarui status attendance_status menjadi 1 (Hadir).

SCREENSHOTS FROM STREAMLIT



Menu

Pilih fitur

Manajemen Internal

Audit Transaksi

Tiket & Order

Database: alumnievents.db

SQL source: uts_dump.sql

Auto-init aktif: database akan dibuat otomatis jika belum ada.

Alumni Events Dashboard

Tiket & Order

Mockup UI/UX

Tiket Saya

Scan Simulator

Pilih QR dari data

QR-3C83C31C

Atau input QR secara manual

QR-3C83C31C

Scan & Update Attendance

participant_id	qr_code	attendance_status	full_name	event_name	start_date
2191	QR-3C83C31C	Belum Hadir	Naufal Mulyani Mulyono	Grand Alumni Homecoming 2027	2027-11-20 18:00:00

Menu

Pilih fitur

Manajemen Internal

Audit Transaksi

Tiket & Order

Database: alumnievents.db

SQL source: uts_dump.sql

Auto-init aktif: database akan dibuat otomatis jika belum ada.

Alumni Events Dashboard

Tiket & Order

Mockup UI/UX

Tiket Saya

Scan Simulator

Simulasi scan QR jika kode valid di-input, attendance_status akan diubah menjadi 1 (Hadir).

Pilih QR dari data

QR-3C83C31C

Atau input QR secara manual

QR-3C83C31C

Scan & Update Attendance

participant_id	qr_code	attendance_status	full_name	event_name	start_date
2191	QR-3C83C31C	Belum Hadir	Naufal Mulyani Mulyono	Grand Alumni Homecoming 2027	2027-11-20 18:00:00

DESCRIPTION

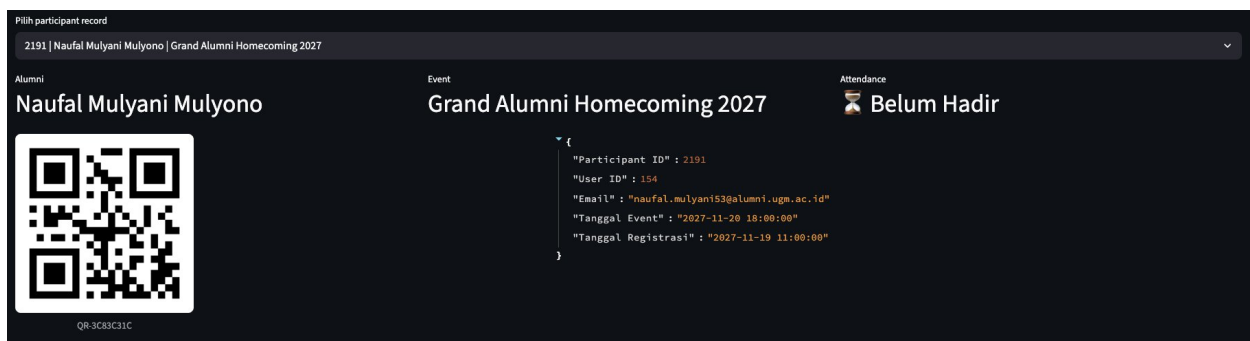
This feature was designed to support a self-service event experience in which one QR code can be used both as an event ticket and as a merchandise pickup identifier. The idea is to make the process more practical for alumni participants while also reducing manual checking by the event staff.

The main data source for this feature is the event_participants table, especially the qr_code and attendance_status columns. The feature is also connected to events and merch_orders so that the selected participant ticket can be associated with a specific event and related order data. This creates a more complete participant view instead of showing only a standalone QR value.

In the code, this feature is implemented in the “Tickets & Orders” menu. The page is divided into several functional parts. First, there is a mockup or UI preview section to illustrate the intended design concept of the ticket page. Second, there is a “My Ticket” section where the user can select a participant record from available ticket data. Third, there is a scan simulation area where a QR code can be entered to simulate check-in.

```
print("==== Preview 3: Tiket Saya (20 participant terbaru) ====")
display(pd.read_sql_query(
    """
    SELECT
        ep.participant_id,
        au.full_name,
        e.event_name,
        ep.qr_code,
        ep.attendance_status,
        ep.registration_date
    FROM event_participants ep
    JOIN alumni_users au ON au.user_id = ep.user_id
    JOIN events e ON e.event_id = ep.event_id
    ORDER BY ep.registration_date DESC, ep.participant_id DESC
    LIMIT 20
    """,
    conn
))
```

When a user selects a ticket record, the application queries the participant and event data from the database, then dynamically generates the QR image from the stored qr_code value. The QR is rendered directly in the Streamlit interface so it behaves like a digital event ticket. In addition to the QR image, the page may also show supporting information such as event name, user identity, registration date, and attendance status.



The scan simulation is the core interactive logic of this feature. In this part, the user inputs a QR code into the system. The code then checks whether that QR exists in the event_participants table. If a matching record is found, the application updates attendance_status from 0 to 1, which means the participant is marked as present. After the database transaction is committed, the app refreshes the displayed data so the updated attendance state becomes visible immediately.

From an implementation perspective, this feature demonstrates both read and write operations. It reads participant and event data to display the ticket, and it writes updated attendance information when a scan is simulated. The QR generation logic is also an important part of the implementation, because the app must convert the stored string into an image format that Streamlit can render correctly.

This feature is valuable because it combines event access and merchandise pickup into one unified workflow. Instead of using separate systems for attendance and order collection, the participant only needs one QR code. This improves efficiency, reduces confusion, and creates a smoother user experience during the event.

CONCLUSION

These three features show how the Streamlit dashboard is not only used for data visualization, but also for operational control and transaction handling. The Board Period Management feature focuses on automated internal data maintenance, the Order Traceability Audit feature focuses on transaction transparency and investigation, and the Tickets & Orders feature focuses on participant self-service and attendance validation.

From a technical perspective, the dashboard integrates Streamlit as the frontend, SQL queries for data access and updates, and structured logic to ensure that each feature is interactive, practical, and directly connected to the database. This makes the application suitable as both an academic project and a prototype for a real administrative event system.

SOURCE CODE

Important Note!

- Put the **vouching.zip** and **database_dump.sql** into the same folder as the **alumni_dashboard_setup.ipynb** and the **app.py** to ensure that the program works.
- You also need to load the database into your DBMS in order for the system to work
- Here is the code link

Here's my folder structure for reference.

